

4. Pandas及其基本使用

在科学计算中，难免会要跟表格数据打交道。以往我们可以用excel处理表格数据，但如果要在科学计算中处理表格数据，一会儿Python一会儿excel很麻烦，我们希望能够有个工具能够直接用编程的方法操纵表格数据，让它实现和excel一样的效果。Pandas正是这样一种工具，它在Numpy的基础上实现了对表格数据的增删改查、统计处理，适合处理大规模表格数据。这一节我们主要学习的内容有：

- [4.1 pandas中的基础数据结构](#)
- [4.2 利用pandas进行数据分析](#)

4.1 Pandas中的基础数据结构

4.1.1 Series

`pandas.Series` 是 `pandas` 库中用于存储一维数组的数据结构，它类似于 `NumPy` 的一维数组 (`ndarray`)，但提供了更丰富的功能，特别是与索引的关联，使得数据操作更为灵活和强大。

创建 `pandas.Series`

你可以使用 Python 列表或 `NumPy` 数组来创建一个 `pandas.Series`。在创建时，可以指定索引 (`index`)，如果不指定，`pandas` 会自动创建一个从 0 开始的整数索引。

```
import pandas as pd
import numpy as np

# 使用列表创建
s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)

# 使用 NumPy 数组创建
s2 = pd.Series(np.array([1, 2, 3, 4, 5]))
print(s2)

# 指定索引
s3 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
print(s3)
```

你也可以使用 Python 字典来创建 pandas.Series，字典的键将作为 Series 的索引，字典的值将作为 Series 的数据。

```
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
s4 = pd.Series(data)
print(s4)
```

基本用法

1. 访问元素

你可以通过索引名或位置来访问 Series 中的元素。

```
# 通过索引名访问
print(s3['a'])

# 通过位置访问（位置索引从 0 开始）
print(s3[0])
```

2. 修改元素

你可以直接通过索引名或位置来修改 Series 中的元素。

```
# 通过索引名修改
s3['a'] = 10
print(s3)

# 通过位置修改
s3[0] = 100
print(s3)
```

3. 索引和切片

Series 支持类似于 Python 列表的切片操作，但索引切片是基于索引名的，而不是位置（除非索引本身就是整数）。

```
# 索引切片
print(s3['a':'c']) # 注意这里是闭区间

# 如果索引是整数，也可以按位置切片
s5 = pd.Series([1, 2, 3, 4, 5], index=[0, 1, 2, 3, 4])
print(s5[0:3]) # 索引为整数的 Series 可以按位置切片
```

4. 运算

Series 支持与标量（单个值）以及另一个 Series 的运算。

```
# 与标量运算
s6 = s3 + 10
print(s6)

# 与另一个 Series 运算（要求索引对齐）
s7 = pd.Series([1, 2, 3], index=['a', 'c', 'e'])
print(s3 + s7) # 在共同索引上进行加法运算
```

常用的属性与方法包括：

- `.index`：查看 Series 的索引。
- `.values`：查看 Series 的值（以 NumPy 数组形式）。
- `.describe()`：快速查看 Series 的统计信息（如计数、均值、标准差等）。
- `.head(n)`：查看 Series 的前 `n` 个元素。
- `.tail(n)`：查看 Series 的后 `n` 个元素。
- `.isnull()` / `.notnull()`：检查 Series 中的元素是否为空（NA/NaN）。
- `value_count()`：统计每个取值的计数。

这些只是 `pandas.Series` 的一些基本用法，`pandas` 提供了大量用于数据清洗、分析、可视化的功能，建议深入学习以充分利用其强大功能。

4.1.2 DataFrame

`DataFrame` 是 `pandas` 库中用于存储和操作结构化数据的主要数据结构，它以表格形式存在，类似于 Excel 或 SQL 表。以下将详细介绍 `DataFrame` 的创建与基本用法。

DataFrame 的创建

1. 使用列表创建

你可以使用嵌套的列表（二维列表）来创建 DataFrame，其中每个内部列表代表一行数据。

```
import pandas as pd

data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
columns = ['A', 'B', 'C']
df = pd.DataFrame(data, columns=columns)
print(df)
```

2. 使用字典创建

你可以使用字典来创建 DataFrame，其中字典的键是列名，字典的值是列表（或数组），表示该列的数据。

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)
print(df)
```

或者，你也可以使用字典的列表来创建 DataFrame，其中每个字典代表一行数据，字典的键是列名。

```
data = [{'A': 1, 'B': 4, 'C': 7}, {'A': 2, 'B': 5, 'C': 8}, {'A': 3, 'B': 6, 'C': 9}]
df = pd.DataFrame(data)
print(df)
```

3. 使用 NumPy 数组创建

NumPy 是 pandas 的底层库之一，你可以使用 NumPy 的二维数组来创建 DataFrame。

```
import numpy as np

data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df = pd.DataFrame(data, columns=['A', 'B', 'C'])
print(df)
```

4. 从文件读取

pandas 提供了多种从文件读取数据并创建 DataFrame 的方法，如 `read_csv`、`read_excel` 等。

```
# 从 CSV 文件读取
df_csv = pd.read_csv('file.csv')

# 从 Excel 文件读取
df_excel = pd.read_excel('file.xlsx')
```

DataFrame 的基本用法

1. 访问数据

- **通过列名访问**：可以直接使用列名作为属性或键来访问 DataFrame 中的列。

```
print(df['A']) # 访问列 A
```

- **通过位置访问**：可以使用 `.iloc[]` 基于整数位置进行访问，使用 `.loc[]` 基于标签进行访问。

```
print(df.iloc[0]) # 访问第一行
print(df.loc[0]) # 如果索引是整数且从 0 开始，这也有效，但通常用于标签索引
```

2. 修改数据

可以直接通过赋值来修改 DataFrame 中的数据。

```
df.loc[0, 'A'] = 10 # 修改第一行，列 A 的值为 10
```

3. 常用属性

- **.shape**：返回 DataFrame 的形状（行数，列数）。
- **.columns**：返回 DataFrame 的列名。
- **.index**：返回 DataFrame 的索引。

```
print(df.shape)
print(df.columns)
print(df.index)
```

4. 数据筛选与排序

- **条件筛选**：可以使用布尔索引来筛选满足条件的行。
- **排序**：可以使用 `.sort_values()` 方法根据一列或多列对数据进行排序。

```
# 条件筛选
filtered_df = df[df['A'] > 5]

# 排序
sorted_df = df.sort_values(by='A')
```

5. 数据聚合与分组

可以使用 `.groupby()` 方法对数据进行分组，并使用聚合函数（如 `sum()`、`mean()` 等）对分组后的数据进行计算。

```
grouped = df.groupby('A').sum() # 假设按列 A 的值分组，并计算每组的和
```

6. 数据合并与连接

可以使用 `pd.concat()`、`pd.merge()` 等方法将多个 DataFrame 合并或连接起来。

```
# 合并
df_combined = pd.concat([df1
```

在pandas的DataFrame中进行复杂的查询与运算，通常涉及到多个条件时，你可以使用布尔索引（Boolean indexing）结合逻辑运算符（如 `&`、`|`）来实现。这些逻辑运算符允许你构建复杂的条件表达式，用于筛选出满足特定条件的行或列。

布尔索引与逻辑运算符

- **&（与）**：用于同时满足多个条件的情况。
- **|（或）**：用于满足多个条件中的至少一个的情况。
- **~（非）**：用于否定条件。

注意，当使用 `&` 和 `|` 进行布尔索引时，需要确保每个条件表达式都被括号括起来，以避免因操作符优先级导致的不正确解析。

假设我们有一个DataFrame `df`，包含以下列： 'A'， 'B'， 'C'。

```
import pandas as pd

# 创建示例DataFrame
data = {
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': ['a', 'b', 'a', 'b', 'c']
}
df = pd.DataFrame(data)
```

示例1：同时满足多个条件

查询列 'A' 大于2且列 'B' 小于40的所有行。

```
# 注意条件表达式用括号括起来
filtered_df = df[(df['A'] > 2) & (df['B'] < 40)]
print(filtered_df)
```

示例2：满足多个条件中的至少一个

查询列 'A' 大于3或列 'C' 等于'b'的所有行。

```
# 注意条件表达式用括号括起来
filtered_df = df[(df['A'] > 3) | (df['C'] == 'b')]
print(filtered_df)
```

示例3：否定条件

查询列 'A' 不大于3的所有行。

```
filtered_df = df[~(df['A'] > 3)]
print(filtered_df)
```

使用 query() 方法

对于更复杂的查询，尤其是当查询条件以字符串形式给出时，你可以使用 `DataFrame.query()` 方法。 `query()` 方法允许你以字符串的形式编写查询条件，这对于从用户输入或其他来源动态生成查询条件特别有用。

```
# 使用query方法进行查询
filtered_df = df.query('A > 2 and B < 40')
print(filtered_df)

# 注意：在query中，条件之间用空格和and/or连接，而不是&|
```

使用 `query()` 方法时，请确保条件字符串中的变量名与 `DataFrame` 中的列名完全匹配，并且注意条件之间的逻辑连接词（ `and` 、 `or` ）前后要有空格。

总之， `pandas` 提供了灵活而强大的工具来处理复杂的查询与运算，通过布尔索引和逻辑运算符，你可以轻松地筛选出满足特定条件的数据。

4.1.3 Timestamp

在 `pandas` 中， `Timestamp` 和 `date_range` 是与时间序列数据处理密切相关的两个重要概念，它们分别用于表示单个时间点和一系列连续的时间点（即时间范围）。

Timestamp

`Timestamp` 是 `pandas` 中用于表示单个时间点（日期和时间）的类。它继承自 Python 的 `datetime.datetime` 类，但提供了更多的功能和灵活性，特别是在与 `pandas DataFrame` 一起使用时。 `Timestamp` 对象可以包含年、月、日、时、分、秒等时间信息，并且支持时区（`tz`）信息。

你可以通过几种方式创建 `Timestamp` 对象：


```
import pandas as pd

# 直接从字符串创建
ts1 = pd.Timestamp('2023-04-01 14:30:00')

# 从 datetime.datetime 对象创建
from datetime import datetime
ts2 = pd.Timestamp(datetime(2023, 4, 1, 14, 30))

# 使用当前时间
ts3 = pd.Timestamp.now()

# 指定时区
ts4 = pd.Timestamp('2023-04-01 14:30:00', tz='UTC')
```

Timestamp 对象支持各种时间相关的操作，如日期和时间的加减、时区转换等。

```
# 日期加减
print(ts1 + pd.Timedelta(days=1)) # 增加一天

# 提取特定部分（年、月、日等）
print(ts1.year, ts1.month, ts1.day)

# 时区转换
print(ts4.tz_convert('Asia/Shanghai'))
```

date_range

date_range 是 pandas 中用于生成固定频率日期时间索引的函数。它返回一个 DatetimeIndex 对象，该对象是一系列连续、等间隔的 Timestamp 对象的集合，非常适合用于时间序列数据的索引。

创建 date_range：

```
# 创建从 2023-01-01 到 2023-01-05 的每日索引
dr = pd.date_range(start='2023-01-01', end='2023-01-05')

# 也可以指定频率，如每小时
hr_dr = pd.date_range(start='2023-01-01 00:00', periods=24, freq='H')

# 带有时区信息的日期范围
tz_dr = pd.date_range(start='2023-01-01', periods=5, freq='D', tz='UTC')
```

`date_range` 通常用于创建时间序列数据的索引，或者作为其他时间序列操作的起点。

```
# 使用 date_range 创建 DataFrame 的索引
df = pd.DataFrame(index=dr, data={'A': range(5)})
print(df)

# 结合 Timestamp 使用，进行日期筛选等操作
filtered_df = df[df.index >= pd.Timestamp('2023-01-03')]
print(filtered_df)
```

4.2 利用pandas进行数据分析

4.2.1 重复，缺失与异常

在使用pandas的DataFrame处理数据时，处理数据重复、数据缺失和异常值是数据清洗和预处理的重要步骤。以下是如何分别处理这些问题的详细指导：

1. 处理数据重复

在DataFrame中，你可能会遇到重复的行，这些行可能是由于数据收集或录入过程中的错误导致的。pandas提供了 `drop_duplicates()` 方法来删除重复的行。

```
import pandas as pd

# 假设df是你的DataFrame
# 删除全部重复的行，仅保留第一次出现的行
df_no_duplicates = df.drop_duplicates()

# 如果你想根据特定的列来判断重复项，可以传入`subset`参数
df_no_duplicates_subset = df.drop_duplicates(subset=['column1', 'column2'])

# 如果你想保留重复项中的最后一次出现的行，可以设置`keep='last'`
df_last_duplicates = df.drop_duplicates(keep='last')
```

2. 处理数据缺失

数据缺失是数据清洗中常见的问题。pandas提供了多种处理缺失数据的方法，包括填充、删除或标记缺失值。

你可以使用 `fillna()` 方法用特定值或前向/后向填充来填充缺失值。

```
# 用0填充所有缺失值
df_filled = df.fillna(0)

# 用前一个有效值填充（前向填充）
df_ffill = df.fillna(method='ffill')

# 用后一个有效值填充（后向填充）
df_bfill = df.fillna(method='bfill')
```

如果缺失值太多或你认为它们对分析没有帮助，你可以选择删除含有缺失值的行或列。

```
# 删除含有任何缺失值的行
df_no_missing_rows = df.dropna()

# 删除含有任何缺失值的列
df_no_missing_cols = df.dropna(axis=1)

# 你可以设置`thresh`参数来指定保留至少有多少个非缺失值的行
df_thresh = df.dropna(thresh=2) # 例如，保留至少有两个非缺失值的行
```

3. 处理异常值

异常值（也称为离群点）是那些与其他数据点显著不同的观测值。处理异常值的方法取决于你的具体数据和你的分析目标。

通常，你可以通过描述性统计（如均值、中位数、标准差）或图形化方法（如箱线图）来识别异常值。这里有一个方法，就是通过分位点判断。计算出上四分位点Q1和下四分位点Q3后我们可以顺势得到一个IQR，异常值就是处在小于 $Q3 - 1.5 \times IQR$ 或大于 $Q1 + 1.5 \times IQR$ 的区段内。形如：

```
# 假设我们有一个数值列
df = pd.DataFrame({'Value': [1, 2, 3, 4, 100]})

# 计算IQR
Q1 = df['Value'].quantile(0.25)
Q3 = df['Value'].quantile(0.75)
IQR = Q3 - Q1

# 识别异常值（例如，小于 $Q1 - 1.5 \times IQR$ 或大于 $Q3 + 1.5 \times IQR$ 的值）
outliers = (df['Value'] < (Q1 - 1.5 * IQR)) | (df['Value'] > (Q3 + 1.5 * IQR))

# 替换异常值（例如，用中位数替换）
median_value = df['Value'].median()
df['Value'][outliers] = median_value
```

一旦识别出异常值，你可以选择删除它们、用均值、中位数或其他统计量来替换它们，或者根据具体情况进行其他处理。

```
# 删除异常值
df_clean = df[~((df['column'] > threshold) | (df['column'] < another_threshold))]

# 用均值替换异常值
df['column'] = df['column'].apply(lambda x: x if x <= threshold else mean)

# 其他处理方法...
```

在处理异常值时，要格外小心，因为异常值有时可能包含重要的信息，简单地删除或替换它们可能会丢失这些信息。因此，在决定如何处理异常值之前，最好先了解数据的背景和异常值的可能原因。

4.2.2 数据的规约

数据的规约 (Normalization 或 Standardization) 是数据预处理中非常重要的一步，它可以帮助改善机器学习模型的性能，尤其是那些对特征尺度敏感的算法。下面将详细介绍两种常见的规约方式：最小-最大规约 (Min-Max Normalization) 和Z-分数规约 (Z-Score Standardization)，并在pandas中实现它们。

1. 最小-最大规约 (Min-Max Normalization)

最小-最大规约通过将原始数据线性变换到新的范围 (通常是[0, 1]) 来规约数据。其公式为：

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

其中， X 是原始数据， $X_{\min}$ 和 $X_{\max}$ 分别是原始数据中的最小值和最大值， X_{norm} 是规约后的数据。

```
import pandas as pd
import numpy as np

# 假设df是你的DataFrame, 'feature'是你想要规约的特征列
df = pd.DataFrame({
    'feature': [1, 2, 3, 4, 5]
})

# 最小-最大规约
df['feature_norm'] = (df['feature'] - df['feature'].min()) / (df['feature'].max() - df['feature'].min())

print(df)
```

2. Z-分数规约 (Z-Score Standardization)

Z-分数规约，也称为标准化，通过将原始数据减去其均值然后除以其标准差来规约数据。其公式为：

$$Z = \frac{X - \mu}{\sigma}$$

其中， X 是原始数据， μ 是原始数据的均值， σ 是原始数据的标准差， Z 是规约后的数据。

```
import pandas as pd

# 假设df是你的DataFrame, 'feature'是你想要规约的特征列
df = pd.DataFrame({
    'feature': [1, 2, 3, 4, 5]
})

# Z-分数规约
df['feature_std'] = (df['feature'] - df['feature'].mean()) / df['feature'].std()

print(df)
```

注意：如果数据的标准差为0（即所有值都相同），则除以标准差会导致除以零的错误。在实际应用中，应该处理这种情况，例如通过添加一个小的常数（如 `epsilon`）到标准差中来避免除以零。

这两种规约方式各有优缺点。最小-最大规约适用于你知道数据的范围并且希望数据被规约到特定区间的情况。而Z-分数规约不依赖于数据的具体范围，更适用于那些假设数据服从正态分布或近似正态分布的情况。选择哪种规约方式取决于你的具体需求和数据的特点。

4.2.3 数据的统计描述

当然，pandas提供了丰富的功能来查看DataFrame中的统计信息、计算统计指标，以及进行分组或透视图分析。下面我将分别介绍这些功能。

1. 查看统计信息

对于DataFrame，你可以使用 `.describe()` 方法来快速查看数值型列的统计信息，包括计数、均值、标准差、最小值、四分位数和最大值。默认情况下，`.describe()` 方法仅对数值型列进行汇总，但你可以通过 `include` 参数指定其他类型（如 `'object'` 或 `'all'`）的列。

```
import pandas as pd

# 假设df是你的DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': ['a', 'b', 'c', 'd', 'e']
})

# 查看数值型列的统计信息
print(df.describe())

# 查看所有类型的列的信息（注意：对于非数值型列，输出可能不如数值型列有用）
print(df.describe(include='all'))
```

2. 计算统计指标

除了 `.describe()` 方法外，pandas还允许你使用聚合函数（如 `mean()`，`std()`，`min()`，`max()`，`quantile()` 等）来计算DataFrame或Series的统计指标。这些函数可以直接应用于整个DataFrame、特定的列或分组后的数据。

```
# 计算列A的均值
mean_A = df['A'].mean()
print(mean_A)

# 计算整个DataFrame的均值（仅数值型列）
mean_df = df.mean()
print(mean_df)

# 计算标准差
std_df = df.std()
print(std_df)
```

3. 分组分析

pandas的 `groupby` 方法允许你根据一个或多个键对DataFrame进行分组，并对每个组应用聚合函数。这是进行复杂数据分析的强大工具。

```
# 假设我们有一个额外的列 'Category'
df['Category'] = ['X', 'Y', 'X', 'Y', 'X']

# 根据 'Category' 列分组，并计算每组的均值
grouped = df.groupby('Category').mean()
print(grouped)

# 你也可以对分组后的数据进行更复杂的操作，比如计算每个组的最大值和最小值
grouped_stats = df.groupby('Category').agg({'A': ['min', 'max'], 'B': 'mean'})
print(grouped_stats)
```

4. 透视图分析

虽然pandas本身没有直接名为“透视图”的函数（像Excel中的PivotTable那样），但你可以通过组合 `groupby`、`pivot`（对于某些特定情况）和 `pivot_table` 方法来达到类似的效果。`pivot_table` 方法特别强大，它允许你指定索引、列和值，并自动处理重复项。

```
# 使用pivot_table创建一个简单的透视图
pivot_table = df.pivot_table(values='A', index='Category', aggfunc='mean')
print(pivot_table)

# 你也可以指定多个索引和列，以及不同的聚合函数
pivot_table_complex = df.pivot_table(values='A', index='Category', columns=df.index, aggfunc='max')
# 注意：上面的代码可能不会按预期工作，因为columns=df.index通常不是有效的用法。
# 你应该指定一个具体的列或列列表作为列参数。

# 一个更实际的例子，假设我们有一个额外的 'Year' 列
df['Year'] = [2020, 2020, 2021, 2021, 2022]
pivot_table_year = df.pivot_table(values='A', index='Category', columns='Year', aggfunc='mean')
print(pivot_table_year)
```

通过这些方法，你可以对DataFrame进行深入的统计分析和数据透视。

扩展阅读

datawhale这边已经对pandas的使用写出了详细的教程，也就是熊猫书：[joyful-pandas](#)